# PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
# FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3
5ta práctica (tipo b)
(Segundo Semestre 2026)

## Indicaciones Generales:

- Duración 1h 50 minutos (Inicio: 8:00 a.m. - Fin: 9:50 a.m.) (Subida y verificación de archivos en PAIDEIA: 9:50 a.m. a 9:55 a.m.) (Salida del Laboratorio: 9:55 a.m. a 10:00 a.m.)
- Se les recuerda que, de acuerdo con el reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Para el desarrollo de toda la práctica debe utilizar el sistema operativo Windows y Java 25.
- Para el desarrollo de las preguntas debe usar IntelliJ.
- Para el desarrollo de la práctica puede usar sentencias SQL o Procedimientos Almacenados.
- Está permitido el uso de Internet (únicamente para consultar páginas oficiales de Oracle y Microsoft). No obstante, está prohibida toda forma de comunicación con otros estudiantes o terceros.
- **NO ESTÁ PERMITIDO EL USO DE MATERIAL DE CONSULTA EN FORMATO DIGITAL.** Todo material de consulta debe ser impreso o escrito a mano.
- Suba a Paideia una carpeta con el siguiente nombre **Lab5_2026_2_CO_PA_PN** donde: CO indica: Código del alumno, PA indica: Primer Apellido del alumno y PN indica: primer nombre. De no colocar este requerimiento se le descontarán 3 puntos de la nota final.
- En el archivo donde se encuentra el método **main** deberá incluir un comentario en el que coloque claramente su nombre y su código, de no hacerlo se le descontarán 0.5 puntos en la nota final.
- Se considerará en la calificación el uso de buenas prácticas de programación (aquellas vistas en clase).
- Al finalizar el laboratorio debe entregar un archivo llamado credenciales.txt con el nombre del servidor, base de datos, usuario y contraseña de su base de datos.

**Puntaje total: 25 puntos** = 20 puntos base + 5 **puntos extra**. Las partes que otorgan puntos extra están indicadas en el enunciado entre paréntesis, por ejemplo **(+N puntos extra)**. Los puntos extra se suman a su puntaje y le permiten compensar puntos perdidos en otras preguntas; la nota final no podrá exceder 20.

| | P1 | P2 | P3 | P4 | Total |
|---|---|---|---|---|---|
| **Puntaje base** | 2 | 6 | 10 | 2 | 20 |
| **Puntos extra** | **4** | **–** | **1** | **–** | **5** |
| **Puntaje máximo** | **6** | **6** | **11** | **2** | **25** |

---

## Cuestionario

- La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en las clases 06, 07, 08, 09 y 10: Conexión a Bases de Datos.
- Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

## Descripción del caso

Una plataforma de música para la comunidad universitaria desea implementar un sistema piloto llamado **SpotiPUCP**, que genere automáticamente playlists personalizadas para sus usuarios a partir de un catálogo de canciones registrado en el sistema. Cada playlist registra el usuario al que pertenece, el nombre de la playlist, su fecha de creación y las canciones que la conforman.

El objetivo principal es generar una playlist para cada usuario registrado, asignándole 10 canciones seleccionadas aleatoriamente del catálogo disponible. Una vez generadas, las playlists deben almacenarse en la base de datos para su posterior consulta y visualización.

En resumen, el sistema debe leer el catálogo de canciones y los usuarios registrados, seleccionar aleatoriamente 10 canciones distintas para cada usuario, registrar cada playlist con sus canciones en el orden asignado y, finalmente, mostrar las playlists registradas con su usuario y sus canciones.

**Tenga en cuenta que el proyecto se llamará spotipucp y que el paquete base de la solución será: pe.edu.pucp.spotipucp**

# Base de datos SpotiPUCP

A continuación, se muestra el modelo de base de datos de **SpotiPUCP**. Se registran los usuarios (código, nombre y correo) para quienes se generarán las playlists y un catálogo de canciones, donde cada canción almacena su título, artista, género y duración en segundos. Cada playlist se asocia a un usuario y almacena su nombre y fecha de creación. La relación entre playlists y canciones se maneja mediante la tabla intermedia **playlist_cancion**, que indica qué canciones fueron asignadas a cada playlist y en qué orden aparecen dentro de ella. El diagrama ER se muestra en la Figura 1.

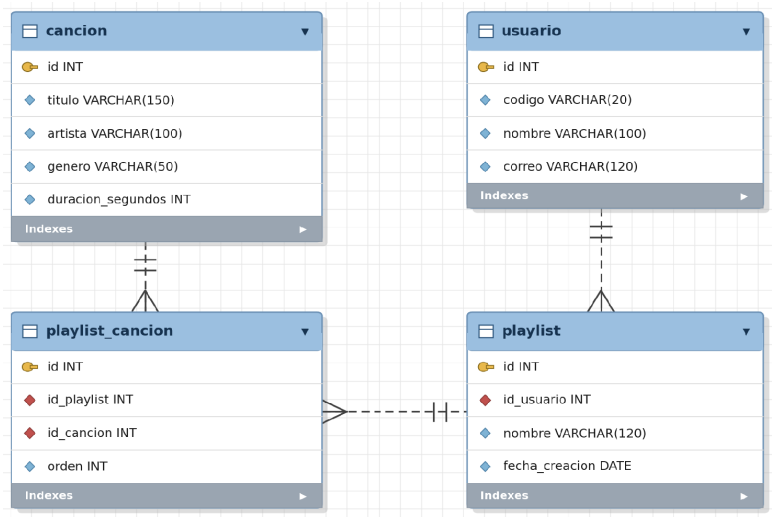



*Figura 1 - Diagrama ER de SpotiPUCP*

Se proporcionará un único script SQL que crea la base de datos en MySQL, la inicializa con el registro de usuarios y el catálogo de canciones e incluye procedimientos almacenados que puede utilizar durante el laboratorio. Usted puede usar o modificar dichos procedimientos y/o crear sus propios procedimientos o sentencias SQL según crea conveniente. **El script deberá ejecutarse antes de iniciar el desarrollo del laboratorio.**

# **Pregunta 01:** Proyecto, capa de modelo y DBManager (2 puntos + 4 extra)

Cree desde cero un proyecto Maven multimódulo llamado **spotipucp**. La cantidad y los nombres de los módulos quedan a su criterio, según la convención de nombrado de su turno, siempre que respete la arquitectura en capas (modelo, conexión a la base de datos, acceso a datos, negocio y aplicación de consola). Se solicita:

1. Crear el proyecto multimódulo con los módulos correspondientes a cada capa y las dependencias necesarias entre ellos.
2. Implementar la **capa de modelo** con las entidades de dominio y sus relaciones, tal como se muestran en la Figura 2. **(+3 puntos extra)**
3. Agregar la dependencia al driver de MySQL (MySQL Connector/J) en el módulo encargado de la conexión a la base de datos.
4. Implementar el **DBManager** con soporte para MySQL, encargado de centralizar la obtención de las conexiones que utilizará la capa de acceso a datos. Los datos de conexión (servidor, base de datos, usuario y contraseña) deben ser leídos de un archivo. **(+1 punto extra)**

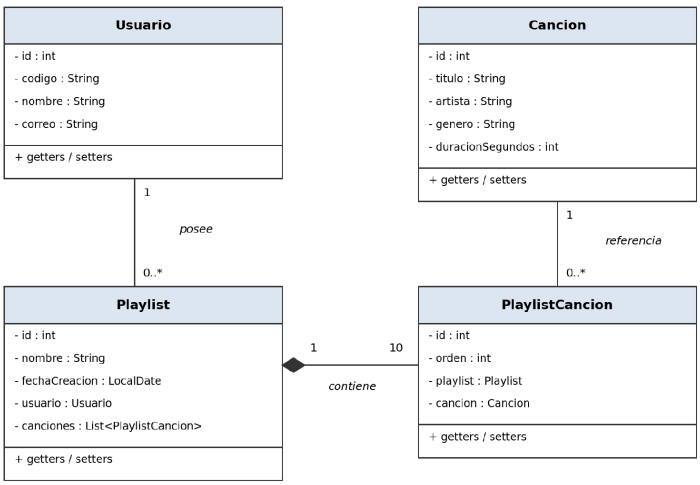



*Figura 2 - Diagrama de clases de la capa de modelo (SpotiPUCP)*

*Los nombres de clases, interfaces y métodos que aparecen en este enunciado son referenciales: puede utilizar el diseño trabajado en su turno del curso, siempre que respete lo solicitado en cada pregunta.*

## **Pregunta 02:** DAOs y lógica no transaccional (6 puntos)

Implemente las operaciones de consulta necesarias para generar las playlists, respetando la arquitectura en capas propuesta. Estas operaciones no requieren manejo transaccional, ya que cada una corresponde a una única operación de lectura sobre la base de datos.

**Capa de acceso a datos:**

1. Implementar el DAO de usuarios con una operación que permita obtener el listado de usuarios registrados.
2. Implementar el DAO de canciones con una operación que permita obtener el listado (catálogo) completo de canciones.

**Capa de negocio:**

1. Implementar en la clase de negocio de usuarios una operación que, apoyándose en el DAO de usuarios, retorne los usuarios para quienes se generarán las playlists.
2. Implementar en la clase de negocio de canciones una operación que, a partir del catálogo obtenido mediante el DAO de canciones, retorne 10 canciones distintas seleccionadas aleatoriamente.

A continuación se muestra un ejemplo referencial de esta última operación.

```
public List<Cancion> seleccionarCancionesAleatorias() {
    List<Cancion> catalogo = // Obtener la listado (catálogo de canciones) de la base de datos

    Collections.shuffle(catalogo);
    return new ArrayList<>(catalogo.subList(0, 10));
}
```

La evaluación considerará la correcta aplicación del patrón DAO, la separación de responsabilidades entre capas y la correspondencia entre los métodos implementados y las operaciones de consulta requeridas.

## **Pregunta 03:** DAOs y lógica transaccional (10 puntos + 1 extra)

Implemente las operaciones de registro y consulta de playlists. A diferencia de la pregunta anterior, el registro de una playlist implica escribir en dos tablas relacionadas (playlist y playlist_cancion), por lo que debe resolverse como una única unidad lógica de trabajo.

**Capa de acceso a datos:**

Implementar los DAOs necesarios para gestionar las playlists del sistema, considerando lo siguiente:

1. Permitir el registro de una nueva playlist asociada a un usuario, incluyendo las canciones que la conforman y el orden que cada una ocupa dentro de ella. Al registrarla, se debe poder recuperar el identificador generado.
2. Permitir la lectura de las playlists registradas, incluyendo su información principal, el usuario asociado y el detalle de sus canciones en el orden correspondiente.

*Las operaciones de escritura de los DAOs deben poder participar en una transacción controlada desde la capa de negocio; es decir, no deben confirmar ni revertir los cambios por su cuenta.*

**TransactionManager: (+1 punto extra)**

Implementar un **TransactionManager** encargado de administrar el ciclo de vida de la transacción y de proveer la conexión que compartirán los DAOs que participan en ella. Debe ofrecer las operaciones necesarias para iniciar una transacción, confirmarla, revertirla y liberar los recursos utilizados.

**Capa de negocio:**

Implementar en la clase de negocio de playlists las siguientes operaciones:

1. **Registro de una playlist (transaccional):** recibe el usuario, el nombre de la playlist y la lista de canciones seleccionadas, y debe apoyarse en el **TransactionManager** para:
   a. Registrar la playlist asociada al usuario, asignando como fecha de creación la fecha actual.
   b. Registrar el detalle de la playlist con las canciones recibidas, asignando a cada una su orden de forma secuencial, del 1 al 10.
   c. Garantizar que ambas operaciones se ejecuten como una única unidad lógica de trabajo: si alguna de ellas falla, no deberá persistir ningún cambio en la base de datos.
   d. Liberar adecuadamente los recursos utilizados, independientemente de si la operación fue exitosa o no.
2. **Consulta de playlists (no transaccional):** retorna las playlists registradas con su usuario y el detalle de sus canciones ordenado, apoyándose en la operación de lectura del DAO.

Se evaluará el manejo de la transacción (inicio, confirmación y reversión) y la consistencia de la información registrada entre la tabla maestra y la tabla detalle.

## **Pregunta 04:** Implementación de la aplicación (2 puntos)

El módulo de la aplicación de consola contendrá una única clase **Programa** con su método **main**, el cual debe generar una playlist para cada uno de los usuarios registrados y, a continuación, mostrar por consola las playlists registradas junto con sus canciones en el orden asignado.

*A continuación se muestra un código de referencia del método main; los nombres utilizados y el manejo de excepciones deben ajustarse a su propia implementación.*

**Programa.java**

```
public class Programa {
  public static void main(String[] args) {
    UsuarioBO usuarioBO = new UsuarioBOImpl();
    CancionBO cancionBO = new CancionBOImpl();
    PlaylistBO playlistBO = new PlaylistBOImpl();

    for (Usuario usuario : usuarioBO.findAll()) {
      List<Cancion> canciones = cancionBO.seleccionarCancionesAleatorias();
      String nombre = "Playlist de " + usuario.getCodigo();
      playlistBO.crearPlaylistConCanciones(usuario, nombre, canciones);
    }

    for (Playlist playlist : playlistBO.findAll()) {
      imprimirPlaylist(playlist);
    }
  }

  private static void imprimirPlaylist(Playlist playlist) {
    System.out.println(playlist.getNombre() + " - " + playlist.getUsuario().getNombre());
    for (PlaylistCancion detalle : playlist.getCanciones()) {
      Cancion cancion = detalle.getCancion();
      System.out.printf("   %d. %s - %s%n",
          detalle.getOrden(), cancion.getTitulo(), cancion.getArtista());
    }
  }
}
```

Profesores del curso:

Freddy Paz  José Corcuera
David Allasi  Eric Huiza

Pando, 23 de septiembre de 2026